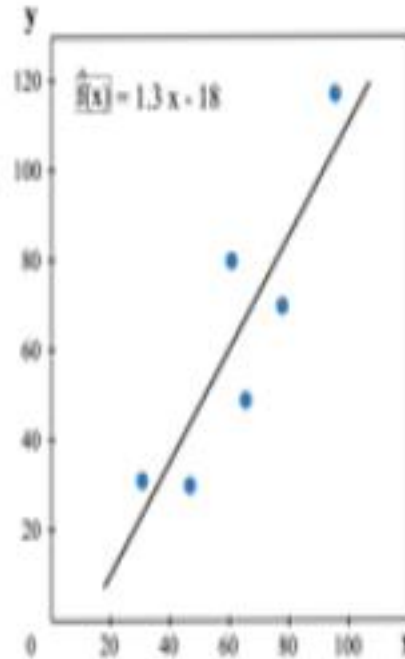


Intro to Data Science

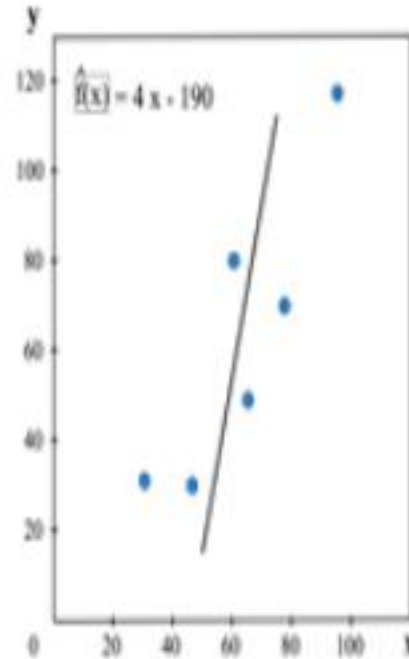
Lec 05

Problem in Linear Regression

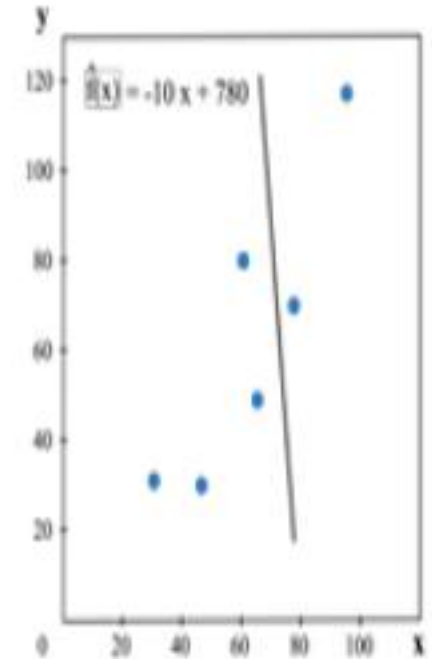
- $y = ax + b$
- Our aim is to find the “best fit line”.
- Change value of “a” and “b”, we obtain different lines.
- How many lines we draw to achieve the “best fit line”.
- it takes exponential time.



(a) $a = 1.3$ and $b = -18$;
 $SSR = 1,248.15$.



(b) $a = 4$ and $b = -190$;
 $SSR = 20,326$.

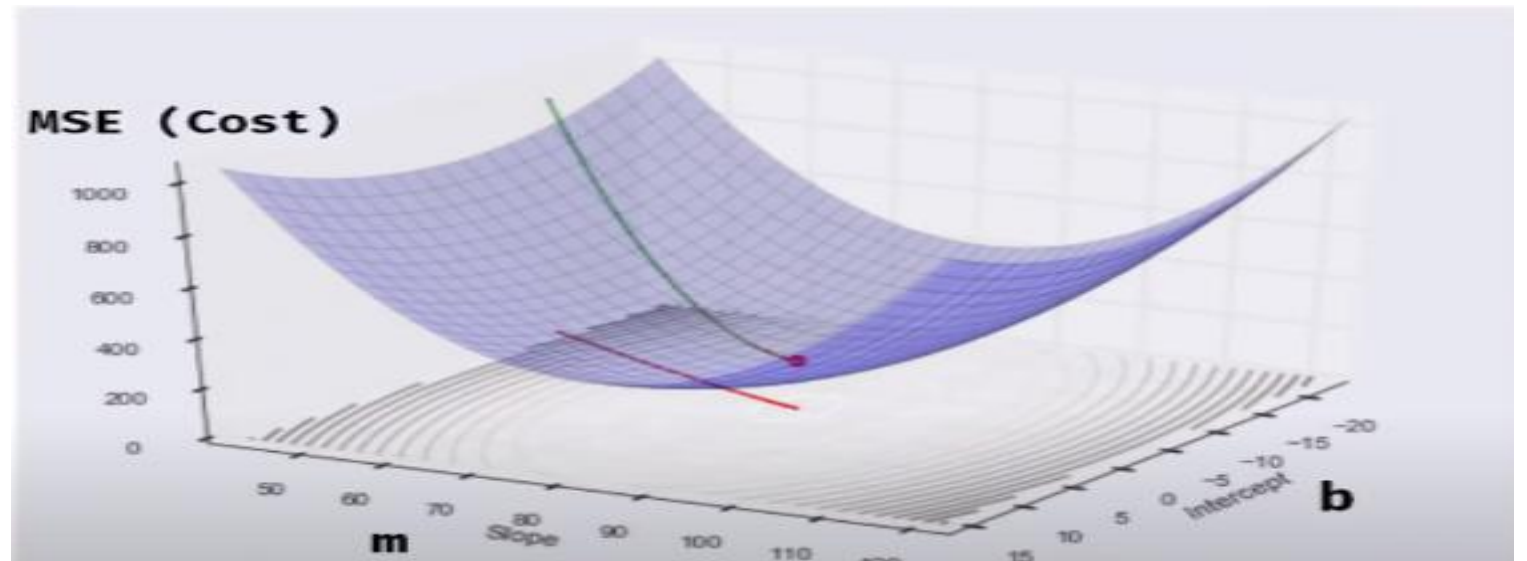


(c) $a = 10$ and $b = 780$;
 $SSR = 388,806$.

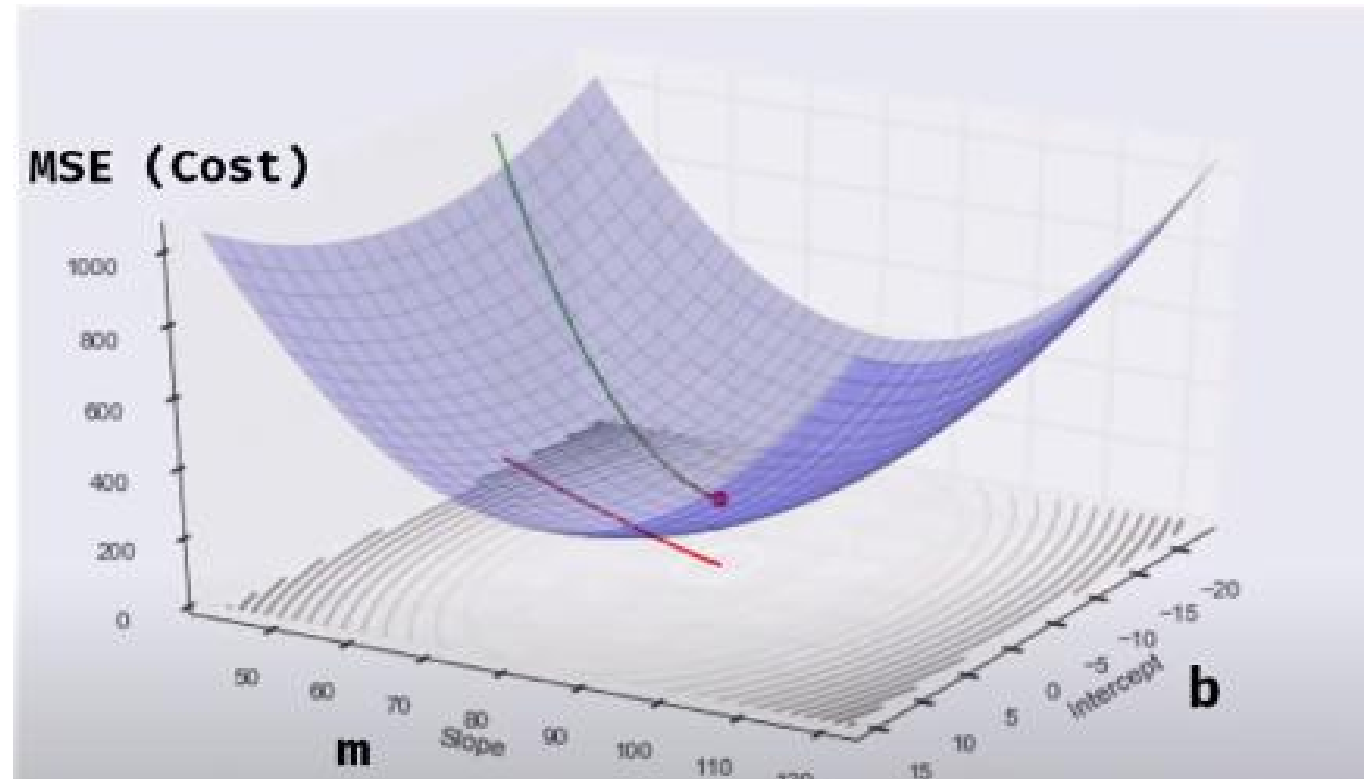
Gradient Decent

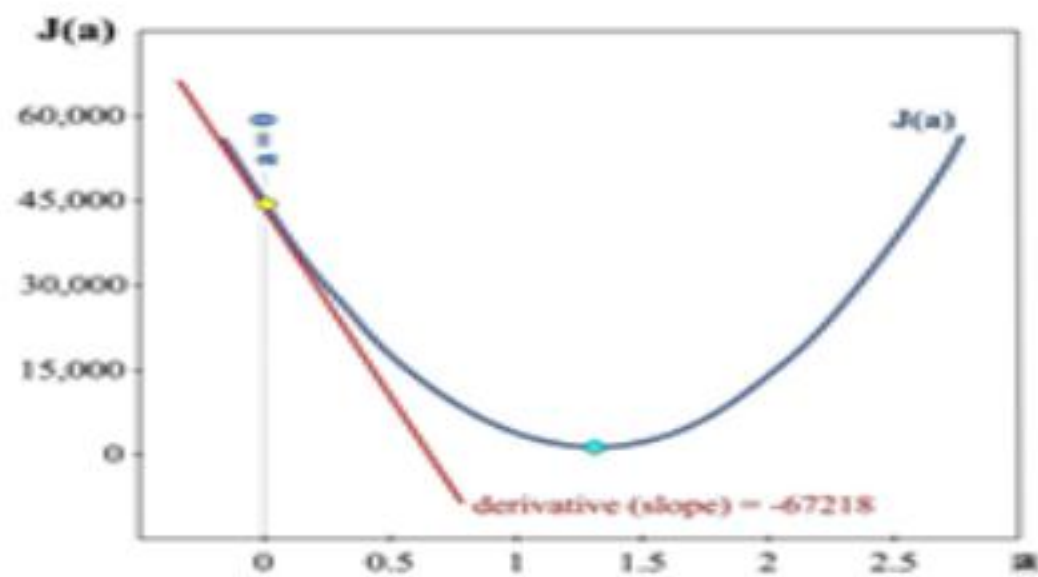
- Gradient Descent is an algorithm that finds the best fit line for given training dataset.
- it help in finding local minima of a function.
- $y=mx+b$, every value of “m” and “b” you find the some cost function.
- it minimize the cost function using iteration

initially start with random values of these parameters and try to find the optimal ones.

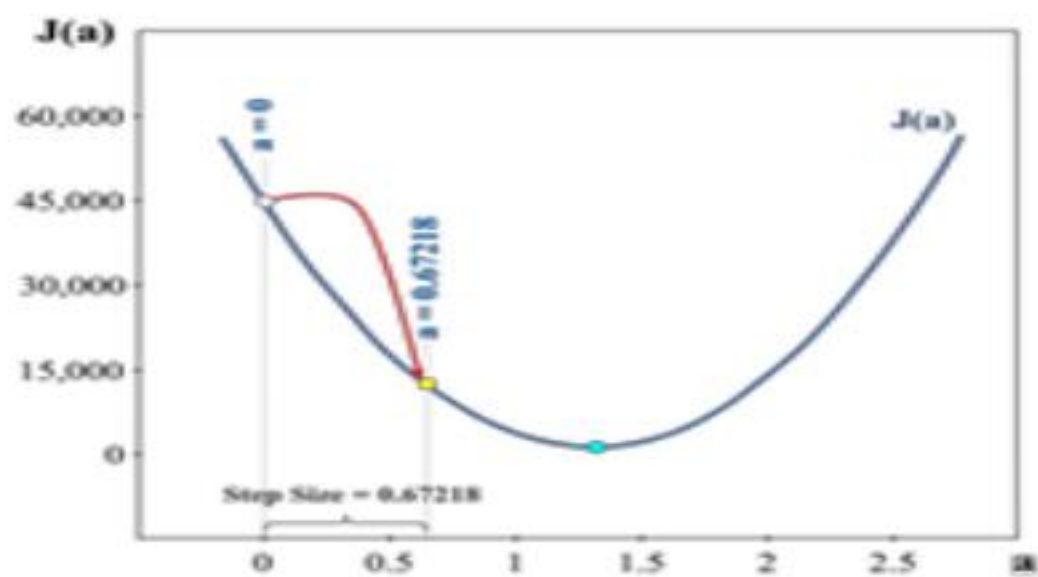


- we find direction.
- Once we find that direction, we will take a baby step in that direction.
 - current position $\rightarrow \rightarrow \rightarrow$ initial parameters
 - baby step $\rightarrow \rightarrow \rightarrow$ learning rate
 - direction $\rightarrow \rightarrow \rightarrow$ partial derivative (gradient)

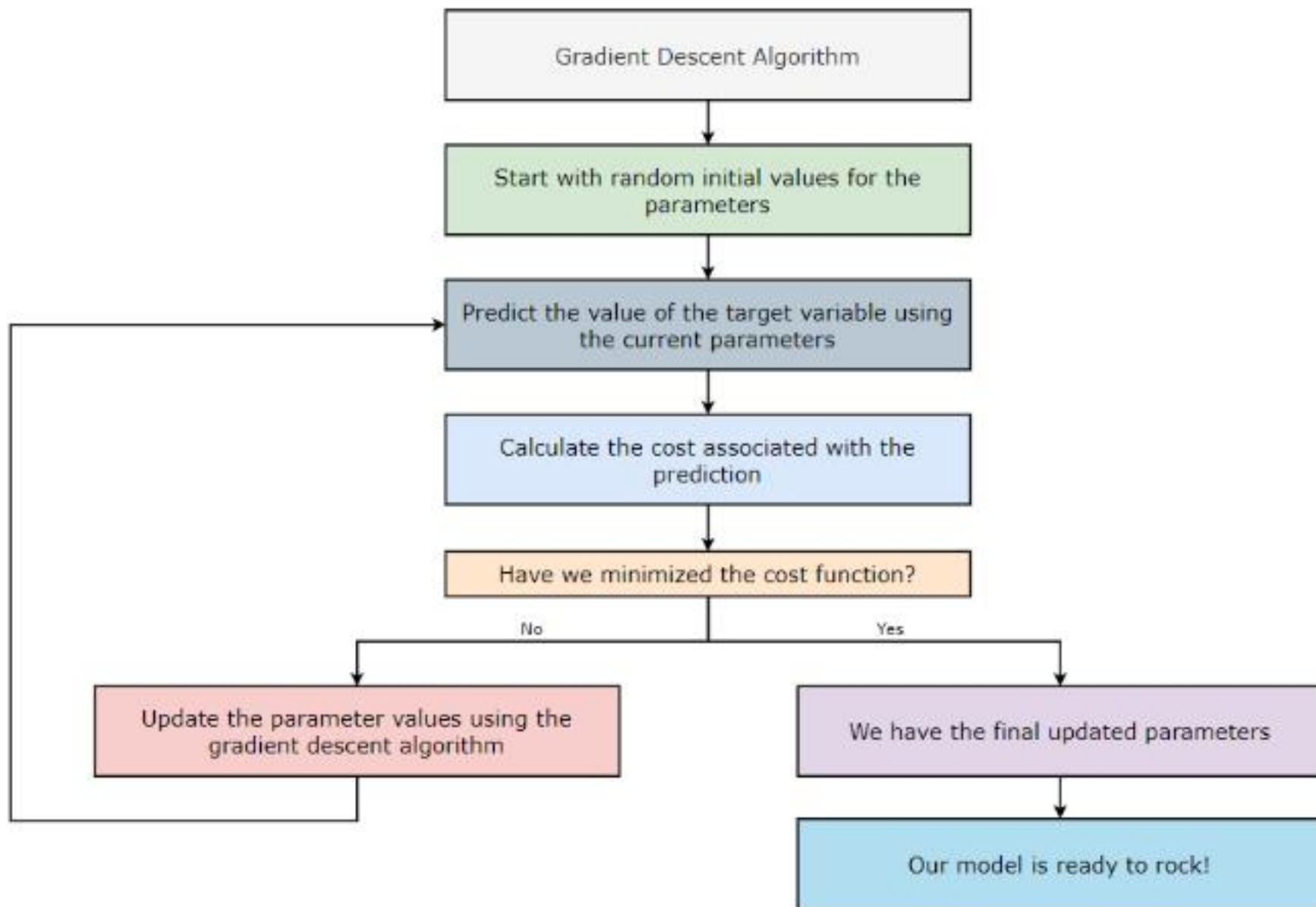




(a) Derivative at the first iteration



(b) Step at first iteration



Algorithm

1. Start with random initial values for the parameters.
2. Predict the value of the target variable using the current parameters.
3. Calculate the cost associated with the prediction.
4. Have we minimized the cost? If yes, then go to step 6. If no, then go to step 5.
5. Update the parameter values using the gradient descent algorithm and return to step 2.
6. We have our final updated parameters.
7. Our model is ready to work.

Formula of the Gradient Descent Algorithm

- $y=mx+c$

- mean square error

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - y_{pre})^2$$

where y_i is actual value and y_{pre} is predicted value

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + c))^2$$

- $\frac{\partial}{\partial m} = \frac{1}{n} \sum_{i=1}^n 2(y_i - (mx_i + c)) (0 - (1 \cdot x_i) + 0) = \frac{1}{n} \sum_{i=1}^n -2x(y_i - (mx_i + c))$

- $\frac{\partial}{\partial c} = \frac{1}{n} \sum_{i=1}^n 2(y_i - (mx_i + c)) (0 - (0 + 1)) = \frac{1}{n} \sum_{i=1}^n -2(y_i - (mx_i + c))$

- $m_{new} = m - \text{learning rate} * \frac{\partial}{\partial m} \quad // \text{ step size}$

- $c_{new} = c - \text{learning rate} * \frac{\partial}{\partial c} \quad // \text{ step size}$

Example 1

YearsExperience(X)	Salary (Y)
1.1	39343
1.3	46205
1.5	37731
2	43525
2.2	39891

Iteration: 3 learning rate : 0.001
m=0 , b=0

Iteration 1:

- Step 1:

$$m=0, \quad c=0$$

- Step 2: (Cost find)

$$\begin{aligned} \text{MSE} &= \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + c))^2 \\ &= 1718423908 \end{aligned}$$

- Step 3: (direction find)

$$\frac{\partial}{\partial m} = \frac{1}{n} \sum_{i=1}^n -2x_i(y_i - (mx_i + c)) = -133900.2$$

$$\frac{\partial}{\partial c} = \frac{1}{n} \sum_{i=1}^n -2(y_i - (mx_i + c)) = -82678$$

- Step 4: (step size)

$$m_{new} = m - \text{learning rate} * \frac{\partial}{\partial m}$$

$$= 0 - 0.001 * (-133900.2) = 133.9002$$

$$c_{new} = c - \text{learning rate} * \frac{\partial}{\partial c}$$

$$= 0 - 0.001 * (-82678) = 82.678$$

YearsExperience (X)	Salary (Y)	$\frac{\partial}{\partial m}$	$\frac{\partial}{\partial c}$
1.1	39343	-86554.6	-78686
1.3	46205	-120133	-92410
1.5	37731	-113193	-75462
2	43525	-174100	-87050
2.2	39891	-175520.4	-79782

- **Iteration 2:**

- **Step 1:**

$$m = 133.9002, \quad c = 82.678$$

- **Step 2: (Cost)**

$$\begin{aligned} \text{MSE} &= \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + c))^2 \\ &= 1693751863 \end{aligned}$$

- **Step 3: (direction find)**

$$\begin{aligned} \frac{\partial}{\partial m} &= \frac{1}{n} \sum_{i=1}^n -2x_i(y_i - (mx_i + c)) \\ &= -132883.0178 \end{aligned}$$

$$\begin{aligned} \frac{\partial}{\partial c} &= \frac{1}{n} \sum_{i=1}^n -2(y_i - (mx_i + c)) \\ &= -82078.8074 \end{aligned}$$

- **Step 4: (step find)**

$$\begin{aligned} m_{\text{new}} &= m - \text{learning rate} * \frac{\partial}{\partial m} \\ &= 133.9002 - 0.001 * (-132883.0178) \\ &= 266.78321 \end{aligned}$$

$$\begin{aligned} c_{\text{new}} &= c - \text{learning rate} * \frac{\partial}{\partial c} \\ &= 82.678 - 0.001 * (-82078.8074) \\ &= 164.756 \end{aligned}$$

Years Experience (X)	Salary (Y)	$\frac{\partial}{\partial m}$	$\frac{\partial}{\partial c}$
1.1	39343	39113.03178	-86048.66992
1.3	46205	45948.25174	-119465.4545
1.5	37731	37447.4717	-112342.4151
2	43525	43174.5216	-172698.0864
2.2	39891	39513.74156	-173860.4629

- **Iteration 3:**

- **Step 1:**

$$m = 266.78321 \quad c = 164.756$$

- **Step 2: (Cost)**

$$\begin{aligned} \text{MSE} &= \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + c))^2 \\ &= 1669448587 \end{aligned}$$

- **Step 3: (direction find)**

$$\begin{aligned} \frac{\partial}{\partial m} &= \frac{1}{n} \sum_{i=1}^n -2x_i(y_i - (mx_i + c)) \\ &= -131873.4718 \end{aligned}$$

$$\begin{aligned} \frac{\partial}{\partial c} &= \frac{1}{n} \sum_{i=1}^n -2(y_i - (mx_i + c)) \\ &= -81484.11043 \end{aligned}$$

- **Step 4: (step find)**

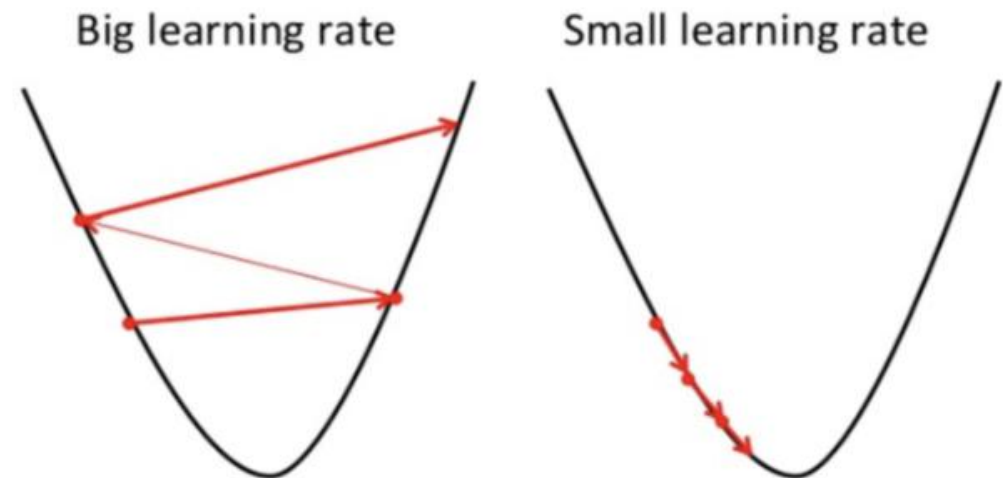
$$\begin{aligned} m_{\text{new}} &= m - \text{learning rate} * \frac{\partial}{\partial m} \\ &= 266.78321 - 0.001 * (-131873.4718) \\ &= 358.6566 \end{aligned}$$

$$\begin{aligned} c_{\text{new}} &= c - \text{learning rate} * \frac{\partial}{\partial c} \\ &= 164.756 - 0.001 * (-81484.11043) \\ &= 246.240 \end{aligned}$$

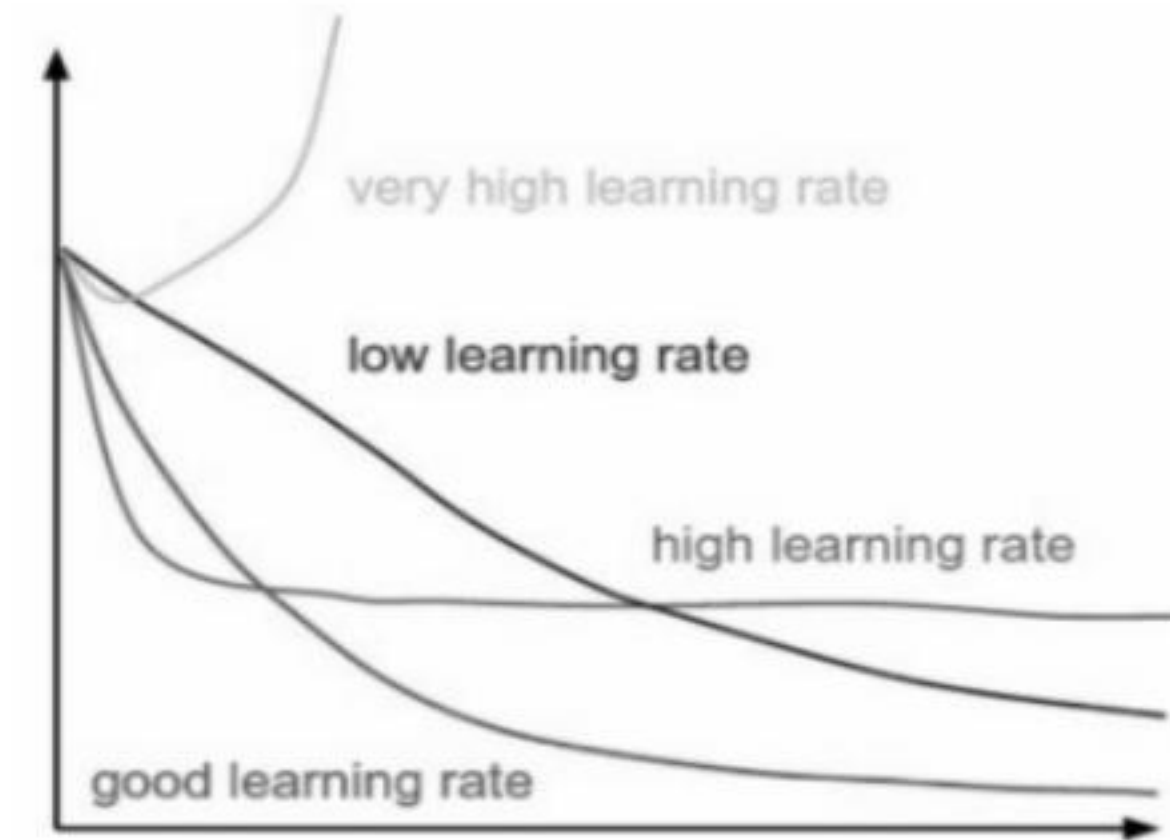
Years Experience(X)	Salary (Y)	$\frac{\partial}{\partial m}$	$\frac{\partial}{\partial c}$
1.1	39343	-85546.52146	77769.56496
1.3	46205	-118802.9072	91386.85168
1.5	37731	-111498.2076	-74332.1384
2	43525	-171306.7104	-85653.3552
2.2	39891	-172213.0122	78278.64192

Learning Rate (α)

- if the learning rate **excessively small or excessively large** will cause the algorithm to **converge slowly or fail to converge** at all.
- if the learning rate is **too small**, will take **unnecessarily small steps** causing the algorithm to take an **excessively long time to reach the optimal value**.
- if the learning rate is **too large**, will take **large steps that jumps back and forth over the optimal value many times**, unnecessarily; this will be very slow and may cause the algorithm not reach optimal at all.



Learning Rate (α)



Types of Gradient Descent

- **Batch Gradient Descent(Gradient Decent):**

- takes all training example at a time.
- find overall cost
- then find gradient decent

- **Stochastic Gradient Descent**

- for each training example
- find its cost
- then find gradient descent

- **Mini Batch gradient descent**

- for each batch in training example
- find its cost
- then find gradient descent

YearsExperience(X)	Salary (Y)
1.1	39343
1.3	46205
1.5	37731
2	43525

Epoch

- Each time we used all training dataset, that is called an Epoch.
- Suppose in our dataset have 4 samples.
- In batch gradient decent you could carried out one pass to complete an Epoch.
- In stocastic gradient descent you could carried out 4 pass to complete an Epoch
- in mini-batch gradient descent, if mini-batch size is=2 then you could carried out 2 pass to complete an Epoch.

YearsExperience(X)	Salary (Y)
1.1	39343
1.3	46205
1.5	37731
2	43525

Batch Gradient Descent	Stochastic Gradient Descent	Mini-Batch Gradient Descent
compute Gradient to whole training sample	compute Gradient to one training sample at a time	We use a batch of a fixed number of training examples which is less than the actual dataset and call it a mini-batch.
used when the dataset is small.	used when the dataset is large.	
Batch Gradient Descent can be used for smoother curves.		
converges directly to minima.		

References

- <https://towardsai.net/p/tutorials/the-gradient-descent-algorithm>
- <https://acrobat.adobe.com/link/review?uri=urn%3Aaaid%3Aascds%3AUS%3Ab7ad98b3-80ec-44cd-9d16-741f83ff2aaa#pageNum=42> (Chap 3)
- <https://www.javatpoint.com/gradient-descent-in-machine-learning>
- <https://vitalflux.com/gradient-descent-explained-simply-with-examples/>
- <https://grabngoinfo.com/gradient-descent-vs-stochastic-gradient-descent-vs-batch-gradient-descent-vs-mini-batch-gradient-descent/>
- <https://www.geeksforgeeks.org/difference-between-batch-gradient-descent-and-stochastic-gradient-descent/>